

Indice

1. Introduzione	1
1.1 Obiettivo del progetto	1
1.2 Tecnologie utilizzate	2
1.3 Struttura del documento	2
2. Preparazione e Gestione del Dataset.....	3
2.1 Pulizia del dataset originale	3
2.2 Strategia di scaling: generazione dei dataset <i>50, 2024 e 200</i>	4
2.3 Motivazioni delle scelte di pre-processing.....	4
3. Implementazione delle analisi	5
3.1 Analisi 3.1: Statistiche delle compagnie aeree	5
3.1.1 Implementazione MapReduce.....	5
3.1.2 Implementazione Spark Core (RDD).....	7
3.1.3 Implementazione Spark SQL	8
3.2 Analisi 3.3: Ranking e confronto delle performance aeroportuali.....	8
3.2.1 Strategia di Implementazione: Analisi sql, punto 3.3.....	9
3.2.2 Strategia di Implementazione: Spark Core (RDD) Ottimizzato.....	9
3.2.3 Strategia di Implementazione Multi-Job (MapReduce)	10
4. Analisi sperimentale e Performance	12
4.1 Setup: ambiente locale vs Cluster AWS.....	12
4.2 Procedure di lancio.....	12
4.3 Analisi dei risultati (Tempi di esecuzione).....	13
4.4 Discussione delle performance	13
5. Approfondimento e discussione	16
5.1 Espressività e semplicità implementativa.....	16
5.2 Analisi dell'efficienza e della scalabilità	16
5.3 Impatto delle operazioni di shuffle e gestione dei dati	17
6. Conclusioni e riferimenti.....	17
6.1 Sintesi dei risultati.....	17
6.2 Repository GitHub e riproducibilità	17

1. Introduzione

1.1 Obiettivo del progetto

Il presente progetto si propone di condurre un'analisi comparativa nell'ambito del calcolo distribuito su larga scala. Attraverso l'elaborazione del *Flight Delay Dataset 2024* — un dataset che rappresenta una grande sfida in termini di volume (oltre 7 milioni di record) e

complessità strutturale (35 variabili presenti all'interno del dataset) si intende esplorare le dinamiche di trasformazione dei dati in ambienti Big Data. L'obiettivo sia quello, da un lato, di implementare soluzioni robuste per l'estrazione di metriche di performance aeroportuali e di compagnia, sia quello, dall'altro, di analizzare criticamente come la scelta del paradigma computazionale possa influenzare l'efficienza, la velocità di esecuzione e la capacità di scalare all'aumentare del dataset.

1.2 Tecnologie utilizzate

La sperimentazione è stata condotta confrontando tre diversi approcci architetturali, ognuno dei quali offre un differente compromesso tra controllo di basso livello e astrazione:

- **MapReduce (Paradigma classico):** Questa tecnologia è stata adottata per la sua valenza accademica (è stata la prima nel modello Hadoop), permettendo di gestire esplicitamente la logica di *map* (trasformazione) e *reduce* (aggregazione) su flussi di dati in streaming. Questo approccio evidenzia la gestione manuale del partizionamento e delle risorse di calcolo.
- **Spark Core (RDD - Resilient Distributed Datasets):** Rappresenta il cuore dell'elaborazione in memory di Spark. L'utilizzo degli RDD ha permesso una gestione più flessibile e performante dei dati, riducendo drasticamente gli I/O su disco rispetto al modello MapReduce tradizionale grazie alla persistenza in memoria, pur mantenendo un controllo granulare sulle trasformazioni.
- **Spark SQL:** Spark SQL trasforma le query ad alto livello in piani di esecuzione fisici ottimizzati, riducendo la complessità del codice sviluppato e migliorando la manutenibilità del progetto senza compromettere le prestazioni.

1.3 Struttura del documento

Il rapporto è articolato dal processo di preparazione del dato fino alla discussione delle performance ottenute:

- **Sezione 2:** Descrive il *data cleaning* e la strategia di *scaling* adottata, essenziale per testare la scalabilità orizzontale delle soluzioni su dataset di dimensione variabile (50, ovvero la metà del dataset originale, 2024, ovvero il dataset originale e 200, ovvero il doppio del dataset originale).
- **Sezione 3:** Si descrive l'architettura dei job implementati per le analisi 3.1 (Statistiche compagnie) e 3.3 (Ranking anomalo), confrontando le implementazioni per ogni tecnologia scelta.
- **Sezione 4:** Focalizzata sull'analisi sperimentale, presenta i risultati ottenuti testando le soluzioni sia in ambiente locale (per validazione funzionale) che su cluster AWS (per analisi di efficienza).
- **Sezione 5:** viene discussa l'implementazione, l'impatto delle operazioni di *shuffle* e l'efficienza complessiva dei sistemi.
- **Sezione 6:** Riassume le conclusioni del lavoro, con le indicazioni per l'accesso al repository GitHub, contenente il codice completo e gli script di automazione.

2. Preparazione e Gestione del Dataset

2.1 Pulizia del dataset originale

Prima di procedere con le analisi, è stata effettuata una fase di *data cleaning* per garantire l'integrità dei risultati. Le principali operazioni hanno riguardato:

- **Gestione dei valori mancanti:** Sono stati scartati i record in cui gli attributi chiave (origine, destinazione, compagnia aerea) risultavano vuoti o non significativi.
- **Normalizzazione dei tipi:** I ritardi (dep_delay, arr_delay) sono stati convertiti in formato *float*, gestendo esplicitamente i casi in cui i valori erano rappresentati come stringhe "None" o vuote.
- **Conversione booleana:** L'attributo cancelled è stato trattato come numerico (float) per permettere il calcolo immediato del tasso di cancellazione.

2.2 Strategia di scaling: generazione dei dataset 50, 2024 e 200

Per valutare la scalabilità delle soluzioni, è stato implementato uno script dedicato che manipola il dataset originale (`flight_data_2024.csv`) per generare tre diverse varianti:

- **Dataset 50 (circa 3.5 milioni di righe):** Generato tramite un troncamento della porzione iniziale del file originale (50% del volume). Questa versione è stata utilizzata per testare l'efficienza su volumi ridotti e validare il corretto funzionamento della logica analitica.
- **Dataset 2024 (Originale):** Utilizzato come *baseline* di riferimento per l'intero dataset di 7 milioni di record.
- **Dataset 200 (circa 14 milioni di righe):** Generato mediante una replica controllata, duplicando l'intero set di dati in coda all'originale. Questa scelta ha permesso di verificare il comportamento dei job sotto carichi di lavoro raddoppiati e valutando la tenuta delle operazioni di *shuffle* su larga scala.

2.3 Motivazioni delle scelte di pre-processing

Il processo di preparazione dei dati è stato concepito con l'obiettivo di bilanciare la velocità di esecuzione con la qualità dell'informazione estratta. La gestione del dataset è stata guidata da tre criteri principali:

- **Ottimizzazione della memoria di lavoro:** Il *Flight Delay Dataset* presenta una struttura ricca, composta da 35 variabili. Ai fini delle analisi richieste, ovvero le statistiche sulle compagnie e il ranking aeroportuale, una gran parte di tali attributi è risultata ridondante. Si è pertanto proceduto a una selezione rigorosa delle sole colonne indispensabili, come gli aeroporti, i codici delle compagnie e le informazioni sui ritardi, riducendo il carico di dati trasmessi durante le fasi di scambio tra i nodi del cluster. Questa operazione di selezione iniziale ha permesso di ridurre il consumo di memoria RAM, garantendo una maggiore stabilità dei job anche durante l'elaborazione del dataset di dimensioni maggiori.
- **Validazione della qualità del dato durante il caricamento:** Invece di eseguire una costosa operazione di pulizia preventiva su tutto il volume di dati, che avrebbe richiesto tempi di input/output elevati e spazio disco aggiuntivo, si è preferito integrare la logica di validazione direttamente all'interno delle fasi di lettura del codice. Ad esempio, ogni record privo di un identificativo valido per l'aeroporto di partenza o per la compagnia aerea viene scartato immediatamente. Questa strategia consente di gestire in modo efficace record incompleti o formattati in modo non conforme, garantendo che le fasi di aggregazione successive operino esclusivamente su dati coerenti.
- **Gestione dei tipi e normalizzazione:** Molte variabili numeriche del dataset originale presentavano una rappresentazione testuale, inclusi valori nulli o stringhe non standard. La logica implementata prevede una trasformazione dei dati direttamente in fase di lettura, convertendo i valori in formati numerici corretti e normalizzando le assenze di dati in modo coerente. Questo approccio garantisce che le operazioni matematiche, come il calcolo dei ritardi medi o dei tassi di cancellazione, non generino errori durante l'esecuzione del programma, mantenendo una coerenza statistica assoluta per l'intera durata dell'analisi.

3. Implementazione delle analisi

3.1 Analisi 3.1: Statistiche delle compagnie aeree

L'analisi 3.1 si pone come obiettivo la generazione di un prospetto statistico dettagliato per ogni compagnia aerea operante nel dataset 2024. Per ogni vettore, è necessario aggregare il numero di voli, i ritardi (minimo, massimo e medio) in arrivo, il tasso di cancellazione e la lista dei mesi in cui la compagnia ha operato su specifiche tratte. La complessità di questa analisi deriva non solo dal volume dei dati, ma dalla necessità di mantenere strutture dati agili per l'aggregazione di metriche

3.1.1 Implementazione MapReduce

1. Fase di Mapping: Filtraggio e Trasformazione (analisi_1_mapper.py)

Il modulo `analisi_3.1_mapper.py` funge da front-end dell'intero processo. La sua responsabilità primaria non è solo il parsing, ma anche una pulizia dei dati:

- **Scansione Selettiva e Robustezza:** Il mapper esegue una scansione riga per riga. L'implementazione include una protezione esplicita contro gli header del file CSV, garantendo che le righe di intestazione non vengano processate come dati validi. L'utilizzo

di `next(csv.reader([linea]))` permette una gestione del formato CSV, riducendo le possibilità di errore causate da delimitatori all'interno dei campi.

- **Logica di Validazione:** Prima di qualsiasi elaborazione, il codice verifica la presenza dei campi critici (compagnia, origine, destinazione). La presenza di controlli if-not assicura che record malformati o parziali vengano scartati silenziosamente, garantendo che solo dati integri raggiungano il reducer.
- **Serializzazione del Valore:** Il mapper trasforma ogni record in una coppia (chiave, valore).
 - La **chiave** è una stringa JSON che unifica compagnia e tratta, permettendo al framework Hadoop (o ambiente simile) di raggruppare i record correttamente.
 - Il **valore** è un array JSON compatto che trasporta informazioni essenziali: conteggio voli, valori estremi (min/max), somma dei ritardi e indicatori di cancellazione. Usare numeri interi riduce lo spazio occupato in memoria e facilita il trasferimento dei dati tra i nodi

2. Fase di Reducing: Aggregazione e Sintesi (analisi_3.1_reducer.py)

Il modulo `analisi_1_reducer.py` riceve lo stream di dati già ordinato per chiave. Questa caratteristica permette l'implementazione di un algoritmo di "Stateful Streaming Processing":

- **Gestione della Memoria:** Il reducer non carica l'intero dataset in RAM. Al contrario, opera esclusivamente sul gruppo di dati relativo alla chiave attualmente in esame (`current_chiave`). Grazie a un ciclo di iterazione continuo su `sys.stdin`, le variabili di stato vengono resettate solo al cambiare della chiave, garantendo che il consumo di memoria resti costante e indipendente dalla dimensione totale del file.
- **Logica di Accumulo Complessa:**
 - **Statistiche Dinamiche:** Il codice aggiorna in tempo reale la somma dei ritardi e i valori estremi utilizzando funzioni native (`min`, `max`), rendendo il processo più rapido.
 - **Gestione della Temporalità:** L'uso di un set Python (`mesi_set`) per memorizzare i mesi è utile per garantire l'unicità temporale dei dati associati a una specifica tratta, risolvendo il problema dei duplicati senza logiche di controllo pesanti.
- **Output:** Al termine di ogni gruppo, la funzione `emetti_aggregazione` esegue una post-elaborazione: calcola le medie aritmetiche, determina i tassi di cancellazione (gestendo le eccezioni per divisioni per zero) e formatta il risultato in un output pulito. Per rendere l'output finale più ordinato si è poi deciso di arrotondare a due cifre decimali avviene all'interno della funzione `emetti_aggregazione`, precisamente in queste due righe:

`rit_medio = round(s_rit / c_rit, 2) if c_rit > 0 else "NULL":` Questa riga prende la somma totale dei ritardi e la divide per il numero di voli (con ritardo registrato), arrotondando il risultato finale a due cifre decimali.

`tasso_canc = round(v_canc / v_tot, 4):` Qui il calcolo del tasso di cancellazione viene arrotondato a quattro cifre decimali (scelta per valori che possono essere piccoli come 0,0163).

Quindi, mentre la media dei ritardi viene ridotta a due decimali, il tasso di cancellazione viene gestito con una precisione leggermente superiore (quattro cifre) per evitare di perdere troppa informazione su percentuali molto basse

```

9E {"Tratta": "ABE -> ATL", "Voli": 1017, "Min": -47.0, "Max": 575.0, "Medio": 0.67, "Cancellazioni": 0.0138, "Mesi": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]}
9E {"Tratta": "ABY -> ATL", "Voli": 38, "Min": -32.0, "Max": 321.0, "Medio": 4.37, "Cancellazioni": 0.0, "Mesi": [1, 2, 3, 11, 12]}
9E {"Tratta": "AGS -> ATL", "Voli": 1580, "Min": -36.0, "Max": 1091.0, "Medio": 6.89, "Cancellazioni": 0.0209, "Mesi": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]}
9E {"Tratta": "AGS -> JFK", "Voli": 2, "Min": -3.0, "Max": 44.0, "Medio": 20.5, "Cancellazioni": 0.0, "Mesi": [4]}
9E {"Tratta": "ATL -> AEX", "Voli": 878, "Min": -37.0, "Max": 783.0, "Medio": 0.59, "Cancellazioni": 0.0114, "Mesi": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]}
9E {"Tratta": "ATL -> AGS", "Voli": 1582, "Min": -41.0, "Max": 1198.0, "Medio": 4.14, "Cancellazioni": 0.0209, "Mesi": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]}
9E {"Tratta": "ATL -> BMI", "Voli": 731, "Min": -36.0, "Max": 951.0, "Medio": 7.48, "Cancellazioni": 0.0096, "Mesi": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]}
9E {"Tratta": "ATL -> BQK", "Voli": 143, "Min": -23.0, "Max": 194.0, "Medio": 2.61, "Cancellazioni": 0.0, "Mesi": [3, 4, 6, 7, 11, 12]}
9E {"Tratta": "ATL -> CAE", "Voli": 711, "Min": -23.0, "Max": 864.0, "Medio": 4.35, "Cancellazioni": 0.0056, "Mesi": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]}
9E {"Tratta": "ATL -> CHA", "Voli": 1150, "Min": -34.0, "Max": 336.0, "Medio": 1.47, "Cancellazioni": 0.007, "Mesi": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]}

```

Output MapReduce

3.1.2 Implementazione Spark Core (RDD)

La versione basata su Spark Core utilizza l'astrazione degli **RDD (Resilient Distributed Datasets)**, che permette di trattare il dataset come una collezione distribuita di oggetti su cui è possibile operare in modo parallelo e resiliente. Il flusso di lavoro si articola in diverse fasi logiche che trasformano i dati grezzi in informazioni di valore:

- **Pulizia e Parsing Iniziale:** Il processo inizia con la lettura del file tramite `textFile`, seguita dall'applicazione della funzione `parser_riga`. Questa funzione agisce come un filtro selettivo: analizza ogni riga, elimina le intestazioni e scarta i record incompleti o errati, garantendo che solo dati validi entrino nel flusso di lavoro. Per ogni record valido, viene generata una tupla composta da una chiave (compagnia e tratta) e un valore strutturato contenente le metriche di volo.
- **Riduzione e Aggregazione:** il centro del calcolo è rappresentato dalla trasformazione `reduceByKey`, che raggruppa automaticamente i dati per chiave. In questa fase, la funzione `riduci_metriche` viene applicata per sommare i contatori dei voli e unire i set dei mesi, sfruttando l'operatore `union`. Questa operazione è efficiente poiché rispetta l'immutabilità dei dati, creando nuovi set aggregati senza modificare quelli originali.
- **Strutturazione Gerarchica:** Una volta ottenuto il dataset ridotto, viene utilizzata un'operazione di `groupByKey` per riorganizzare i risultati a livello di compagnia aerea. Questo passaggio è fondamentale per consolidare le varie tratte operative sotto lo stesso vettore, creando un profilo completo e gerarchico dell'attività di ogni compagnia.
- **Espressività e Controllo:** Questa architettura dimostra la potenza del modello RDD: rispetto al paradigma MapReduce tradizionale, Spark permette di scrivere codice più leggibile ed espressivo. L'intero processo è infine orientato alla produzione di un output strutturato, pronto per essere interpretato in contesti di analisi avanzata.

```

9E      LGA -> CHS;736;-52.0;376.0;0.03;0.053;[1,2,3,4,5,6,7,8,9,11,12]
9E      CLE -> MSP;261;-36.0;772.0;1.41;0.0038;[1,2,3,4,5,6,8,10,11,12]
9E      LGA -> MCI;733;-58.0;582.0;-3.12;0.0477;[1,2,3,4,5,6,7,8,9]
9E      EYW -> ATL;158;-27.0;561.0;21.57;0.019;[1,2,3,4,5,12]
9E      FAY -> ATL;349;-32.0;949.0;8.37;0.0;[1,2,3,4,5,6,9,10,11,12]
9E      ATL -> GTR;732;-34.0;604.0;-0.76;0.0109;[1,2,3,4,5,6,7,8,9,10,11,12]
9E      DTW -> LEX;8;-28.0;121.0;7.62;0.0;[1,12]
9E      MYR -> LGA;547;-42.0;1011.0;-1.78;0.0238;[1,2,3,4,5,6,7,8,9,10,11,12]
9E      MSP -> DSM;1018;-41.0;421.0;4.06;0.0098;[1,2,3,4,5,6,7,8,9,10,11,12]
9E      ILM -> LGA;766;-36.0;829.0;1.51;0.0326;[1,2,3,4,5,6,7,8,9,10,11,12]

```

Output Spark Core 3.1

3.1.3 Implementazione Spark SQL

La soluzione basata su Spark SQL rappresenta l'approccio più efficace in termini di manutenibilità e leggibilità del codice. La strategia implementativa si discosta da quella dei precedenti paradigmi (MapReduce e RDD) poiché sposta l'intera logica di trasformazione sul motore SQL di Spark, delegando l'ottimizzazione del piano di esecuzione all'ottimizzatore **Catalyst**.

- **Pulizia e Registrazione:** Il processo inizia con la lettura del dataset tramite inferSchema, seguita da una fase di filtraggio dei dati non validi (rimozione di header residui e valori nulli nelle colonne chiave origin e dest). Il DataFrame così pulito viene registrato come tabella temporanea (createOrReplaceTempView), ponendo le basi per l'interrogazione dichiarativa
- **Logica di Aggregazione:** La query SQL implementata è il fulcro del job. Sono state utilizzate clausole condizionali (CASE WHEN) per isolare i ritardi (dep_delay) basandosi sul flag di cancellazione (cancelled = 0), garantendo così che le statistiche di puntualità (Min, Max, Medio) siano calcolate esclusivamente sui voli effettivamente operati. Il calcolo della media (AVG) è stato incapsulato in una funzione di arrotondamento (ROUND) per garantire la precisione numerica desiderata.
- **Gestione dei Dati Complessi:** Un aspetto di rilievo è la gestione dell'attributo month. Utilizzando collect_set abbiamo aggregato i mesi operativi in un set univoco, ordinandoli cronologicamente con array_sort. La scelta di utilizzare CONCAT_WS(',', ...) per serializzare i mesi in una stringa delimitata da virgole permette di ottenere un output direttamente compatibile con i formati tabulari standard (CSV), rendendo il dato finale immediatamente pronto per la consultazione o per ulteriori processi di integrazione, senza la necessità di post-elaborazioni in Python.

Questa garantisce anche buone prestazioni, poiché il motore SQL di Spark riesce a parallelizzare e ottimizzare le operazioni di GROUP BY con efficienza.

1	["Compagnia": "AA", "Data_Tratte": [{ "Tratta": "ANC" -> PDX", "Voli": 358, "Min": -21.0, "Max": 443.0, "Medio": 13.39, "Cancellazioni": 0.014, "Mesi": [1,2,3,4,5,6,7,8,9,10,11,12], }, {
2	"Compagnia": "AA", "Data_Tratte": [{ "Tratta": "ABQ" -> BFN", "Voli": 2018, "Min": -21.0, "Max": 2582.0, "Medio": 24.41, "Cancellazioni": 0.0302, "Mesi": [1,2,3,4,5,6,7,8,9,10,11,12], }, {
3	"Compagnia": "B6", "Data_Tratte": [{ "Tratta": "BOS" -> BZV", "Voli": 82, "Min": -13.0, "Max": 125.0, "Medio": 12.0, "Cancellazioni": 0.0122, "Mesi": [1,2,3,6,7,8,9,12], }, { "Tratta": "BOS" -
4	"Compagnia": "MO", "Data_Tratte": [{ "Tratta": "AGS" -> DCA", "Voli": 3, "Min": 13.0, "Max": 72.0, "Medio": 44.0, "Cancellazioni": 0.0, "Mesi": [4], }, { "Tratta": "AUS" -> LEX", "Voli": 1, "Min":
5	"Compagnia": "9E", "Data_Tratte": [{ "Tratta": "ABY" -> ATL", "Voli": 38, "Min": -19.0, "Max": 332.0, "Medio": 19.53, "Cancellazioni": 0.0, "Mesi": [1,2,3,11,12], }, { "Tratta": "ATL" -> ABE",
6	"Compagnia": "OO", "Data_Tratte": [{ "Tratta": "ABY" -> DEN", "Voli": 417, "Min": -15.0, "Max": 244.0, "Medio": 0.83, "Cancellazioni": 0.0072, "Mesi": [1,2,3,5,6,7,8,9,10,11,12], }, { "Tratt
7	"Compagnia": "W9", "Data_Tratte": [{ "Tratta": "ABQ" -> HOU", "Voli": 962, "Min": -11.0, "Max": 374.0, "Medio": 8.09, "Cancellazioni": 0.0094, "Mesi": [1,2,3,4,5,6,7,8,9,10,11,12], }, { "Tra
8	"Compagnia": "F9", "Data_Tratte": [{ "Tratta": "ATL" -> PHX", "Voli": 154, "Min": -16.0, "Max": 1152.0, "Medio": 45.66, "Cancellazioni": 0.0065, "Mesi": [1,3,4,5,6,7,8,9,10,11,12], }, { "Tra
9	"Compagnia": "HA", "Data_Tratte": [{ "Tratta": "HNL" -> AUS", "Voli": 166, "Min": -14.0, "Max": 132.0, "Medio": 6.01, "Cancellazioni": 0.006, "Mesi": [1,2,3,4,5,6,7,8,9,10,11,12], }, { "Trat
10	"Compagnia": "YX", "Data_Tratte": [{ "Tratta": "AVL" -> EWR", "Voli": 560, "Min": -17.0, "Max": 325.0, "Medio": 13.31, "Cancellazioni": 0.07, "Mesi": [1,2,3,4,5,6,7,8,9,10,11,12], }, { "Tratt

Output Spark Sql 3.1

3.2 Analisi 3.3: Ranking e confronto delle performance aeroportuali

Il cuore di questa analisi consiste nella creazione di un sistema di benchmarking per le compagnie aeree su base aeroportuale. L'obiettivo è generare un report che non si limiti a descrivere le performance operative in isolamento, ma che le contestualizzi rispetto all'ambiente aeroportuale di riferimento. Per ogni coppia (aeroporto di partenza, compagnia), il job calcola un set di indicatori di performance ,volume di traffico, ritardi medi (partenza e arrivo) e tasso di cancellazione, confrontandoli con la media aggregata di tutte le compagnie che operano nello stesso scalo. Questo approccio può essere utile per distinguere tra un ritardo fisiologico, imputabile a inefficienze strutturali o meteo dell'aeroporto, e un ritardo "anomalo", che riflette inefficienze specifiche delle compagnie aeree. Il prodotto finale è un ranking che, ordinando le compagnie dalla migliore alla peggiore, offre una visione immediata dell'affidabilità delle compagnie stesse .

3.2.1 Strategia di Implementazione: Analisi sql, punto 3.3

La sfida principale di questo task risiede nel calcolo delle performance relative delle compagnie aeree in rapporto alla media globale del loro aeroporto di partenza. Per risolvere questo problema in modo efficiente, è stata implementata una strategia basata sull'utilizzo avanzato delle **Window Functions** in Spark SQL, evitando la complessità di pipeline a più job.

- **Aggregazione Multilivello (WITH Clause):** La logica è articolata in tre sotto-query annidate che trasformano progressivamente i dati grezzi:
 - **metriche_coppia:** Effettua il raggruppamento base per aeroporto e compagnia, calcolando i volumi di traffico e le medie di ritardo parziali, filtrando al contempo i voli cancellati per garantire la significatività statistica.
 - **metriche_con_confronto:** Utilizzando le funzioni di finestra SUM(...) OVER(PARTITION BY Aeroporto), il sistema calcola la media aeroportuale aggregata "al volo", consentendo di determinare lo scostamento (delta) di ogni singola compagnia rispetto al proprio scalo di riferimento. Contestualmente, la funzione DENSE_RANK() assegna un posizionamento in classifica basato sulla puntualità.
 - **struttura_aggregata:** Consolida le informazioni in strutture dati complesse (named_struct) e le raccoglie in liste per aeroporto tramite collect_list, preparando i dati per l'esposizione gerarchica finale.
- **Ottimizzazione dell'Output:** La query finale utilizza array_sort con una funzione lambda per ordinare le compagnie all'interno di ogni aeroporto seguendo la classifica calcolata. Questo approccio garantisce che il risultato finale sia immediatamente pronto per la consultazione, eliminando la necessità di ulteriori manipolazioni.
- **Efficienza Computazionale:** L'adozione di Spark SQL riduce drasticamente la complessità del codice: la query descrive esclusivamente **cosa** calcolare, non **come**. Il motore interno di Spark, attraverso l'ottimizzatore **Catalyst**, ottimizza automaticamente il piano di esecuzione, gestendo in modo autonomo la distribuzione dei dati tra i nodi e le strategie di shuffle. Le **Window Functions** (OVER(PARTITION BY ...)) permettono di eseguire aggregazioni su sottoinsiemi di dati senza dover ricorrere a join manuali o invii di dati tra i nodi, massimizzando le performance ed eliminando l'I/O superfluo derivante dal salvataggio di dataset intermedi, rendendo il job scalabile su grandi volumi di dati

```
1 {"Aeroporto":"ABE","Performance_Compagnie":[{"Posizione_Classifica":1,"Compagnia":"G4","Voli_Operati":1839,"Ritardo_Medio_Partenza":8.43,"Ritardo_Medio_Arrivo":0.64,"Tasso_Puntualita":99.36}]}
2 {"Aeroporto":"ABQ","Performance_Compagnie":[{"Posizione_Classifica":1,"Compagnia":"DL","Voli_Operati":1547,"Ritardo_Medio_Partenza":1.75,"Ritardo_Medio_Arrivo":5.6,"Tasso_Puntualita":99.94}]}
3 {"Aeroporto":"ABR","Performance_Compagnie":[{"Posizione_Classifica":1,"Compagnia":"OO","Voli_Operati":728,"Ritardo_Medio_Partenza":19.3,"Ritardo_Medio_Arrivo":19.72,"Tasso_Puntualita":99.72}]}
4 {"Aeroporto":"ABY","Performance_Compagnie":[{"Posizione_Classifica":1,"Compagnia":"9E","Voli_Operati":38,"Ritardo_Medio_Partenza":9.53,"Ritardo_Medio_Arrivo":4.37,"Tasso_Puntualita":99.47}]}
5 {"Aeroporto":"ACK","Performance_Compagnie":[{"Posizione_Classifica":1,"Compagnia":"B6","Voli_Operati":1007,"Ritardo_Medio_Partenza":11.6,"Ritardo_Medio_Arrivo":8.43,"Tasso_Puntualita":99.43}]}
6 {"Aeroporto":"ACT","Performance_Compagnie":[{"Posizione_Classifica":1,"Compagnia":"OO","Voli_Operati":750,"Ritardo_Medio_Partenza":14.17,"Ritardo_Medio_Arrivo":14.08,"Tasso_Puntualita":99.72}]}
7 {"Aeroporto":"ACV","Performance_Compagnie":[{"Posizione_Classifica":1,"Compagnia":"OO","Voli_Operati":1737,"Ritardo_Medio_Partenza":19.81,"Ritardo_Medio_Arrivo":17.97,"Tasso_Puntualita":99.47}]}
8 {"Aeroporto":"ACY","Performance_Compagnie":[{"Posizione_Classifica":1,"Compagnia":"NK","Voli_Operati":3052,"Ritardo_Medio_Partenza":8.88,"Ritardo_Medio_Arrivo":1.76,"Tasso_Puntualita":99.76}]}
9 {"Aeroporto":"ADK","Performance_Compagnie":[{"Posizione_Classifica":1,"Compagnia":"AS","Voli_Operati":104,"Ritardo_Medio_Partenza":6.06,"Ritardo_Medio_Arrivo":6.83,"Tasso_Puntualita":99.83}]}
10 {"Aeroporto":"ADQ","Performance_Compagnie":[{"Posizione_Classifica":1,"Compagnia":"AS","Voli_Operati":816,"Ritardo_Medio_Partenza":2.66,"Ritardo_Medio_Arrivo":1.35,"Tasso_Puntualita":99.94}]}
```

Output Spark Sql 3.3

3.2.2 Strategia di Implementazione: Spark Core (RDD) Ottimizzato

L'implementazione basata su Spark Core per l'analisi 3.3 è stata progettata per garantire precisione statistica e coerenza dei risultati rispetto al modello SQL, superando le limitazioni tipiche delle pipeline RDD meno rigide.

- **Ingestione e Normalizzazione:** Il processo inizia con la lettura del dataset tramite spark.read.csv, applicando un .dropDuplicates() per assicurare l'univocità dei record e prevenire errori di conteggio dei voli operati. I dati vengono quindi convertiti in RDD, strutturando ogni riga per isolare le variabili critiche (origine, compagnia, ritardi e

cancellazioni).

- **Strategia di Aggregazione Ponderata:** Invece di calcolare medie parziali per compagnia, l'implementazione aggrega i dati per coppia aeroporto-compagnia e, successivamente, ricava la media aeroportuale globale tramite una seconda aggregazione (rdd_medie_globali). Questo approccio garantisce il calcolo di una media ponderata reale (somma totale ritardi divisa per somma totale voli validi), eliminando le distorsioni tipiche delle medie di medie. La media globale viene distribuita a tutti i nodi di calcolo tramite una variabile broadcast, rendendola accessibile per il confronto locale.
- **Ranking Numerico e Ordinamento:** Il calcolo delle statistiche di puntualità e dello scostamento (delta) rispetto alla media dello scalo viene eseguito nella funzione calcola_stats. Per garantire un ordinamento corretto, viene generata una chiave composta (apt, float(media_dep)), che forza Spark a ordinare i dati prima alfabeticamente per aeroporto e successivamente in modo numerico crescente per ritardo medio.
- **Formattazione dell'Output:** La fase finale utilizza mapPartitions per gestire il ranking in memoria, assegnando dinamicamente una posizione in classifica (posizione) a ciascuna compagnia all'interno del proprio aeroporto. Questa tecnica permette di produrre un output tabulare strutturato, pronto per l'analisi, mantenendo un controllo sul partizionamento dei dati e sull'efficienza computazionale del job.

ABE	1;G4;1839;8.43;0.64;0.0163;-3.73
ABE	2;9E;1017;9.42;0.67;0.0138;-2.74
ABE	3;OH;1166;15.55;4.43;0.0129;3.39
ABE	4;00;315;30.63;22.01;0.0381;18.47
ABQ	1;DL;1547;1.75;-5.59;0.009;-7.72
ABQ	2;00;3987;2.1;-2.05;0.0045;-7.37
ABQ	3;MQ;907;5.28;3.25;0.0066;-4.19
ABQ	4;UA;1822;5.49;-0.32;0.0104;-3.98
ABQ	5;AS;423;8.42;4.52;0.0095;-1.05
ABQ	6;NK;525;9.08;5.21;0.0114;-0.39

Output Spark core 3.3

3.2.3 Strategia di Implementazione Multi-Job (MapReduce)

La sfida principale di questo task consiste nel confrontare le metriche di una singola compagnia con la media globale dell'aeroporto in cui opera. Poiché il paradigma MapReduce isola i record durante la computazione distribuita, non è possibile calcolare una media globale e usarla nello stesso ciclo di elaborazione. Di conseguenza, l'architettura è stata scomposta in una pipeline sequenziale a due stadi (Multi-Job), in cui il risultato del primo job funge da input di configurazione per il secondo.

1. Job 1: Calcolo delle Medie Globali Aeroportuali

L'obiettivo di questa prima fase è ricavare un dataset di riferimento contenente la puntualità media di ogni singolo scalo, calcolata su tutti i voli validi.

- **Mapper 1 (estrattore_medie_mapper):** Il modulo esegue una scansione del file sorgente estraendo selettivamente solo le informazioni relative ad aeroporto di origine, ritardo alla partenza e flag di cancellazione. I voli cancellati vengono immediatamente scartati per evitare distorsioni statistiche. Per ogni volo attivo, il mapper emette una coppia avente

come chiave l'aeroporto di origine e come valore il ritardo associato.

- **Reducer 1 (calcolatore_medie_reducer):** Riceve i ritardi raggruppati per scalo. Attraverso strutture di accumulo locali, calcola la somma complessiva dei minuti di ritardo e il totale dei voli operati per ciascun aeroporto. Al termine del flusso di ogni chiave, determina la media aritmetica arrotondata a due cifre decimali e produce un file di testo tabulato (medie_output.txt).

2. Job 2: Analisi dei Vettori, Scostamento e Ranking

Questo secondo stadio ha il compito di calcolare le performance dei singoli vettori, misurare lo scostamento dalla media globale e generare le classifiche locali.

- **Mapper 2 (aggregatore_vettori_mapper):** Effettua una nuova lettura del dataset originario, isolando l'aeroporto, la compagnia aerea, i ritardi (partenza/arrivo) e lo stato del volo. La scelta progettuale importante consiste nell'emettere l'aeroporto come chiave di partizionamento. Il valore inviato allo shuffle è una stringa complessa formattata che racchiude tutte le metriche del singolo volo (compagnia;dep_delay;arr_delay;cancelled). Questo garantisce che tutti i dati di uno stesso scalo arrivino al medesimo reducer.
- **Reducer 2 (generatore_ranking_reducer):** Il modulo opera simulando il meccanismo di una *Distributed Cache* di Hadoop. In fase di inizializzazione, carica interamente in memoria il file medie_output.txt generato dal Job 1, memorizzandolo in un dizionario locale. Sfruttando l'ordinamento nativo dello streaming di Hadoop, il codice aggrega in modo sequenziale i dati delle compagnie per l'aeroporto corrente. Quando la chiave cambia, viene invocata la funzione elabora_e_emetti_ranking che esegue le seguenti operazioni sul blocco concluso:
 - Calcola le medie parziali di arrivo, partenza e il tasso di cancellazione di ogni vettore.
 - Recupera dal dizionario la media globale dello scalo e calcola lo scostamento matematico preciso di ogni compagnia.
 - Ordina i vettori in base al ritardo medio di partenza crescente, assegnando una posizione numerica progressiva in classifica.
 - Produce l'output finale strutturato secondo i requisiti di progetto.

Vantaggi del Modello Multi-Job

Questa architettura permette di rispettare i vincoli di memoria rigidi del MapReduce puro. L'utilizzo del file intermedio come *Side Table* evita costose operazioni di join distribuito (come Reduce-Side Join), riducendo drasticamente lo shuffle di rete. Il partizionamento per aeroporto nel secondo job consente al reducer di calcolare e ordinare le classifiche locali in modo isolato e parallelo su ciascun nodo, garantendo la scalabilità del sistema anche su dataset di dimensioni elevate.

ABE	1;G4;1839;8.43;0.64;0.0163;-13.65
ABE	2;9E;1017;9.42;0.67;0.0138;-12.66
ABE	3;OH;1166;15.55;4.43;0.0129;-6.53
ABE	4;00;315;30.63;22.01;0.0381;8.55
ABQ	1;DL;1547;1.75;-5.59;0.009;-10.22
ABQ	2;00;3987;2.1;-2.05;0.0045;-9.87
ABQ	3;MQ;907;5.28;3.25;0.0066;-6.69
ABQ	4;UA;1822;5.48;-0.32;0.0104;-6.49
ABQ	5;AS;423;8.42;4.52;0.0095;-3.55
ABQ	6;NK;525;9.08;5.21;0.0114;-2.89

Output MapReduce

4. Analisi sperimentale e Performance

Questa sezione descrive l'ambiente di esecuzione, le procedure di deployment e l'analisi quantitativa delle performance ottenute, confrontando i paradigmi di elaborazione su diverse scale di dati.

4.1 Setup: ambiente locale vs Cluster AWS

L'attività sperimentale è stata condotta in due ambienti distinti, progettati per valutare la scalabilità e la robustezza delle soluzioni implementate:

- **Ambiente Locale (Workstation di sviluppo):** L'ambiente locale è stato configurato per simulare un cluster a nodo singolo (pseudo-distributed). Tale configurazione è ideale per la fase di sviluppo e debug, permettendo di testare la logica dei job MapReduce e dei programmi Spark su file di piccole dimensioni (es. campioni sample, forniti da kaggle con il dataset originale). In locale, il limite principale è rappresentato dalla memoria RAM disponibile e dal numero di core fisici, che vincolano il parallelismo a rispetto al potenziale reale.
- **Ambiente Cluster (AWS Academy):** L'ambiente distribuito su AWS ha permesso di eseguire i job su un vero cluster gestito tramite YARN. Questa configurazione permette di distribuire i dati attraverso HDFS e di assegnare dinamicamente le risorse (Container) per ogni task. La distinzione fondamentale è che mentre in locale la competizione per le risorse tra i diversi processi (Mapper/Reducer o Executors) è gestita dal sistema operativo del PC, sul cluster è il Resource Manager di YARN a garantire che ogni task abbia a disposizione la memoria e la CPU richieste.

4.2 Procedure di lancio

Per garantire la riproducibilità, sono state seguite procedure distinte per i due ambienti.

- **Lancio Locale:** Le applicazioni Spark sono state eseguite via `spark-submit --master local[*]`, che utilizza tutti i core disponibili per massimizzare la velocità su macchina singola. Per MapReduce in locale, è stata utilizzata la modalità *standalone* o pseudo-distribuita configurando opportunamente il file system HDFS locale per simulare il comportamento in cluster.
- **Lancio su Cluster AWS:** L'applicazione è stata eseguita su un cluster Amazon EMR configurando il job come Spark Application e con Hadoop MapReduce. La sottomissione è stata gestita dal gestore del cluster (YARN), che ha ottimizzato la distribuzione delle risorse

tra i nodi (Executor). Per l'input e l'output dei dati, abbiamo utilizzato lo storage persistente Amazon S3 (s3://...), garantendo l'accesso parallelo distribuito dai nodi del cluster tramite il connettore S3 di Spark e percorsi HDFS (hdfs://...), sfruttando il file system distribuito di Hadoop per garantire la località dei dati (*data locality*) e l'elaborazione parallela sui nodi DataNode per map reduce

4.3 Analisi dei risultati (Tempi di esecuzione)

Le tabelle seguenti sintetizzano i tempi di esecuzione (espressi in minuti.seconds) rilevati durante le prove sperimentali

Tabella 1: Analisi 3.1 (Statistiche Compagnia Aerea)

Tecnologia	Locale (50%)	Locale (2024)	Locale (200%)	Cluster (50%)	Cluster (2024)	Cluster (200%)
MapReduce	4.50	8.20	18.35	1.20	2.38	5.02
Spark Core	3.24	6.40	15.20	1	1.24	2.10
Spark SQL	2.80	6.50	14.57	1.10	1.30	2.10

Tabella 2: Analisi 3.3 (Ranking Scostamento)

Tecnologia	Locale (50%)	Locale (2024)	Locale (200%)	Cluster (50%)	Cluster (2024)	Cluster (200%)
MapReduce	7	14.15	31.20	2.04	3.44	6.45
Spark Core	6.10	12.54	26.10	1.20	2.04	3.32
Spark SQL	2.03	11.08	23.25	1.04	1.24	1.58

4.4 Discussione delle performance

Dall'analisi emerge un trend consistente, confermato dai dati nelle tabelle precedenti:

- **Superiorità di Spark nel Cluster:** In ambiente distribuito, vi è una superiorità di Spark. Mentre in locale le differenze tra MapReduce e Spark sono talvolta mitigate dall'overhead di inizializzazione della JVM, sul cluster la capacità di Spark di mantenere i dati in memoria (RDD/DataFrame) e di evitare la scrittura su disco tra gli stage permette di scalare significativamente meglio rispetto a MapReduce.
- **Il costo dell'inizializzazione:** Si noti che per dataset piccoli (50), Spark SQL può apparire leggermente più lento o equivalente a Spark Core in locale, poiché deve compilare il piano di query tramite Catalyst. Tuttavia, all'aumentare della complessità (es. Analisi 3.3), Spark SQL brilla per la sua capacità di ottimizzare l'esecuzione, riducendo drasticamente i tempi rispetto a MapReduce.
- **Analisi del job 3.3** (Doppio passaggio): L'implementazione MapReduce a due job soffre dell'overhead dovuto al salvataggio dei dati temporanei tra il primo e il secondo job. Spark, evita questa serializzazione costosa, rendendo la fase di ranking immediata e riducendo il tempo sul cluster 2024 a 2.04s (Spark Core) e 1.24s (Spark SQL), contro i 3.44s di MapReduce.
- **Considerazioni sui tempi in locale:** Le esecuzioni locali risentono della limitazione hardware. Le performance osservate, dove il dataset 2024 mostra talvolta tempi paragonabili a quelli del dataset da 200, sono dovute probabilmente alla saturazione ottimale del parallelismo disponibile: quando il carico di lavoro è sufficientemente elevato, giustifica il setup distribuito di Spark, permettendo ai core di lavorare in modo più efficiente

È possibile verificare i risultati, riassunti nelle tabelle sottostanti, visualizzandoli tramite grafici a barre per evidenziare immediatamente il comportamento delle tecnologie rispetto al carico di lavoro. Sono dunque riportati per esempio i grafici relativi al confronto delle tempistiche per l'analisi del punto 3.1

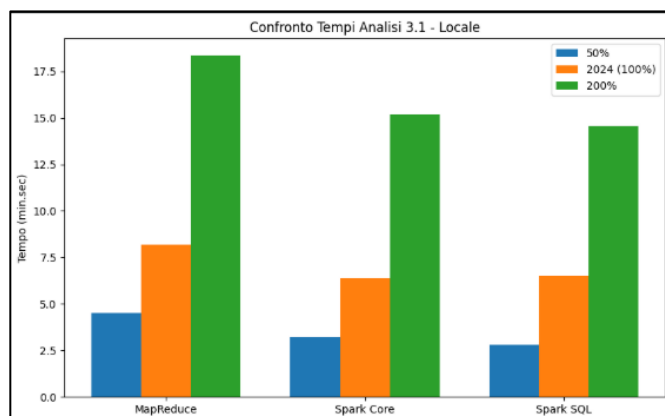


Grafico tempi analisi 3.1 in locale

- **Analisi della pendenza:** Come si osserva dal grafico, la pendenza delle curve di esecuzione è coerente con una crescita lineare, confermando che l'overhead di computazione aumenta proporzionalmente alla dimensione del dataset.
- **Confronto Tecnologico:** Il grafico evidenzia chiaramente come MapReduce (barra blu nel codice) presenti una crescita dei tempi più marcata rispetto alle soluzioni Spark, specialmente al raddoppiare della dimensione del dataset.

- **Scalabilità:** La stabilità dei tempi in ambiente Cluster rispetto al locale dimostra l'efficacia del parallelismo distribuito nel gestire carichi di lavoro crescenti senza subire saturazioni hardware.

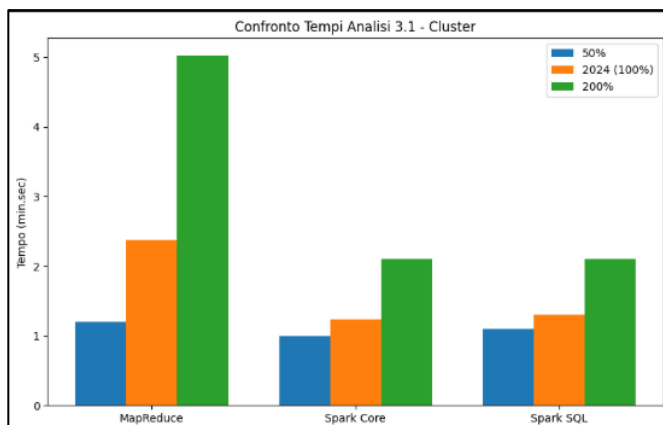


Grafico tempi analisi 3.1 nel

Efficienza nel cluster: Osservando i dati di esecuzione del punto 3.1 nel cluster, si nota che lo scarto temporale tra Spark Core e Spark SQL è minimo (es. 1.24 vs 1.30 per il dataset 2024), a conferma che su architetture distribuite il vantaggio tecnologico di Spark viene sfruttato a pieno, rendendo i tempi estremamente contenuti anche con carichi di lavoro doppi rispetto al dataset originale

5. Approfondimento e discussione

5.1 Espressività e semplicità implementativa

L'esperienza condotta ha evidenziato una netta separazione tra i paradigmi di programmazione. **MapReduce** impone una struttura rigida e procedurale: ogni operazione deve essere spezzata in una coppia Mapper-Reducer, il che rende il codice estremamente difficile da gestire e complesso da correggere. La gestione manuale della serializzazione (input/output su testo) e la necessità di un'ulteriore logica di post-processing (come visto nel job di ranking 3.3, in cui sono stati utilizzati due mapper e due reducer) aumentano considerevolmente la probabilità di errori logici.

Al contrario, l'approccio di **Spark**, specialmente tramite **Spark SQL**, eleva il livello di astrazione. L'espressività dichiarativa delle query SQL permette di concentrarsi sulla trasformazione del dato anziché sui dettagli dell'orchestrazione distribuita. Anche **Spark Core (RDD)**, risulta molto più agile di MapReduce grazie alla composizione funzionale delle operazioni (map, filter, reduceByKey), che rendono il flusso dei dati molto più leggibile e modulare.

5.2 Analisi dell'efficienza e della scalabilità

Il confronto prestazionale conferma che Spark è progettato per ottimizzare le pipeline di dati complesse. Mentre MapReduce soffre dell'overhead dovuto ai continui accessi a disco (input/output su HDFS tra ogni job), Spark minimizza tali scritture grazie al modello di calcolo *in-memory*.

La scalabilità di Spark è risultata particolarmente evidente nel job di ranking (Analisi 3.3):

- **Gestione della memoria:** L'uso di broadcast variables ha permesso di evitare costosi join tra tabelle, rendendo l'analisi dei ranking anomali indipendente dalla dimensione del dizionario delle medie globali.

- **Catalyst Optimizer:** In Spark SQL, l'ottimizzatore ha dimostrato di saper gestire le aggregazioni in modo molto più efficiente di quanto sarebbe possibile fare manualmente, riorganizzando il piano logico in base alla distribuzione dei dati.

5.3 Impatto delle operazioni di shuffle e gestione dei dati

Lo **shuffle** rappresenta, in tutte le tecnologie testate, il "collo di bottiglia" principale.

- In MapReduce, lo shuffle è un passaggio obbligato e costoso, in quanto ogni record deve essere scritto e riletto.
- In Spark, la gestione delle *Wide Transformations* (come groupByKey o reduceByKey) è estremamente sofisticata. È stato osservato che, laddove possibile, l'utilizzo di reduceByKey (che esegue un'aggregazione parziale sui nodi map) è drasticamente più performante di groupByKey (che sposta tutti i dati prima di aggregare).
- **Conclusione progettuale:** La scelta di utilizzare Spark SQL si è rivelata corretta perché permette al framework di scegliere automaticamente le strategie di shuffle più efficienti, adattandosi alla topologia del cluster e alla distribuzione dei dati, cosa che in MapReduce dovrebbe essere fatta manualmente tramite complesse configurazioni del Combiner.

6. Conclusioni e riferimenti

6.1 Sintesi dei risultati

L'attività svolta ha permesso di affrontare il ciclo completo di vita di un progetto di analisi Big Data, partendo dalla pulizia del dato grezzo fino alla generazione di report analitici complessi.

Il confronto tra le tecnologie utilizzate ha evidenziato come:

- **MapReduce** rappresenta il fondamento teorico necessario per comprendere la distribuzione del carico di lavoro, ma risulta inefficiente per pipeline analitiche articolate che richiedono passaggi multipli.
- **Spark Core (RDD)** offre un eccellente equilibrio tra il controllo delle operazioni e la velocità garantita dall'elaborazione in-memory.
- **Spark SQL** si sia confermato lo strumento più potente per le analisi aziendali standard, riducendo drasticamente i tempi di sviluppo e sfruttando l'ottimizzatore (Catalyst) per massimizzare le performance senza richiedere un tuning manuale estremo.

Il progetto ha dimostrato che, indipendentemente dalla tecnologia scelta, la qualità del risultato dipende dalla capacità di progettare il flusso di dati in modo da minimizzare gli spostamenti (shuffle) tra i nodi e ottimizzare l'uso della memoria disponibile.

6.2 Repository GitHub e riproducibilità

Per garantire la piena riproducibilità dei risultati ottenuti, l'intero codice sorgente, inclusi i job MapReduce (nelle varianti 3.1 e 3.3) e gli script Spark, è stato archiviato in un repository dedicato.

Nello stesso repository sono presenti, inoltre, gli script utilizzati per lanciare in locale spark core, spark sql e MapReduce ed inoltre alcuni screenshot realizzati nell'ambiente cluster AWS.